

Aqualis (Staking)

SMART CONTRACT

Security Audit

Performed on Contracts:

Aqualis Staking.sol

Aqualis StakingReward.sol

AQLToken.sol

Platform

POLYGON

hashlock.com.au

JUNE 2024

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Behaviours	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Audit Findings	12
Centralisation	24
Conclusion	25
Our Methodology	26
Disclaimers	28
About Hashlock	29

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Aqualis team partnered with Hashlock to conduct a security audit of their AQLToken.sol, AqualisStaking.sol and AqualisStakingReward.sol smart contracts and their respective tests. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed contracts were secure.

Project Context

Aqualis is a DeFi dApp built on BNB Chain and Metis Network that allows you to stake your AQL tokens and earn rewards based on the lock time.

Simply put, it is a unique time-weighted staking protocol and users can be exponentially rewarded for commitment to the long-term vision of Aqualis.

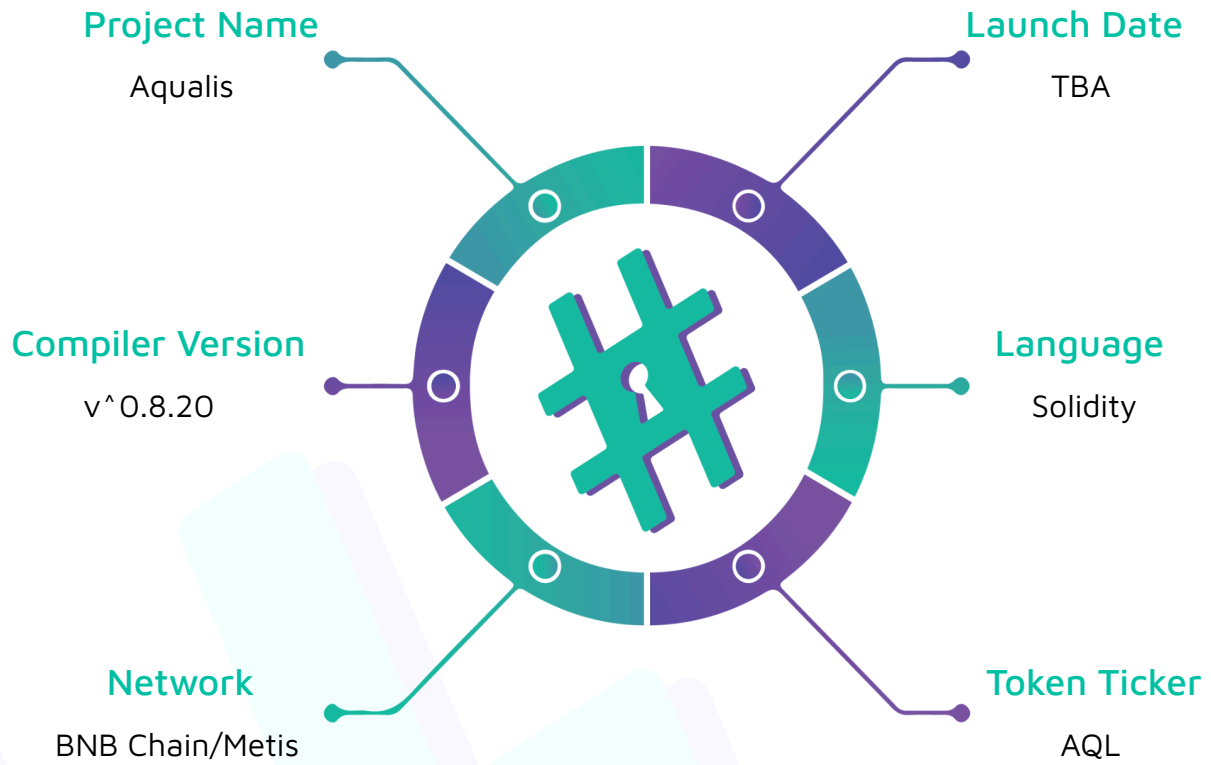
Project Name: Aqualis

Compiler Version: ^0.8.20

Website: www.aqualis.io

Logo:



Visualised Context:

Project Visuals:

Unlock DEX Liquidity with the Novel AMU AI Algorithm

The Aqualis asset multi-utilization AI (AMU AI) algorithm enables up to 80% of DEX liquidity to be simultaneously used as capital for crypto backed loans, drastically increasing lending availability in DeFi without sacrificing trading liquidity.

Aqualis is a multi-functional DeFi Protocol

Keep Assets in Crypto

Gain access to immediate liquidity from crypto holdings without selling, providing users with the funds they need without compromising their market exposure.



Optimized Asset Utilization

Increased efficiency delivers low fees and high deposit rates by maximizing asset usage through the Aqualis AMU AI algorithm, the backbone of the entire Aqualis ecosystem.



AQL Staking

Be part of a unique time-weighted staking protocol and be exponentially rewarded for commitment to the long-term vision of Aqualis.



Audit scope

We at Hashlock audited the solidity code within the Aqualis project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Aqualis Protocol Smart Contracts
Platform	Polygon / Solidity
Audit Date	June, 2024
Contract 1	AQLToken.sol
Contract 1 MD5 Hash	ec8a3eb05e84a3e20ab8264dd2a84cfe
Contract 2	AqualisStaking.sol
Contract 2 MD5 Hash	dd03b32decda7b35b7444fbc4d5d06e6
Contract 3	AqualisStakingReward.sol
Contract 3 MD5 Hash	30e882149746a9dee349ee2f29f78b9f
GitHub Commit Hash	f86a559c3168bf98a372552451413380f58131dc

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. The protocol uses a list of Open Zeppelin contracts. We have identified some significant vulnerabilities that need to be addressed prior to launch.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section.

All vulnerabilities we have identified have been resolved or acknowledged.

Hashlock found:

2 High-severity vulnerabilities

2 Medium-severity vulnerabilities

5 Low-severity vulnerabilities

3 Gas Optimisations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour
AQLToken.sol - ERC20 Token	Contract achieves this functionality.
AqualisStaking.sol - Allows users to: - Stake their AQL tokens and lock them for certain amounts of weeks - Calculate AP (reward multiplier) of users based on the staked amount and lock duration - Increase their staked amount - Increase their lock duration - Withdraw their stake early with a penalty - Withdraw their stake after their lock has ended	Contract achieves this functionality.
AqualisStakingReward.sol - Contract that handles user rewards - Allows users to: - Claim their rewards - View how much reward they have earned	Contract achieves this functionality.

Code Quality

This Audit scope involves the smart contracts of the Aqualis project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring is required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Aqualis projects smart contract code in the form of GitHub access.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments help understand the overall architecture of the protocol.

Dependencies

Per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry-standard open-source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies

Audit Findings

High

[H-01] AqualisStaking.sol::increaseStakingAmount - New lock time calculation does not work as intended

Description

Users with an active stake can call `increaseStakingAmount` function in order to increase their staked amount. Based on this staked amount, compared to the user's already existing stake, the function should increase the lock time of the user's stake. However, the calculation for the lock time does not work as intended.

Vulnerability Details

The `increaseStakingAmount` function updates the `newLockTime` which is used to calculate the new lock time of users staked tokens.

```
function increaseStakingAmount(uint256 _amount) external {
    require(_amount > 0, "Amount must be > 0");
    address staker = _msgSender();
    Stakes memory stake = stakers[staker];
    require(stake.amount > 0, "Stake does not exist");

    aqualisToken.transferFrom(staker, address(this), _amount);

    /// @dev increment stake amount
    stake.amount += _amount;

    uint32 remainingLockTime = stake.unlockAt - uint32(block.timestamp);
    uint32 initialStakeTime = uint32(stake.weeksNum) * 1 weeks;

    /// @dev new Lock Time = (initialStakeTime - remainingLockTime) * portion of
    additional amount +
    /// remainingLockTime
```

```

uint32 newLockTime =
    uint32((initialStakeTime - remainingLockTime) * (_amount / stake.amount))
+ remainingLockTime;

/// @dev rounded-up to weeks
uint8 newLockTimeInWeeks = uint8(newLockTime / 1 weeks + 1);

stake.unlockAt = uint32(block.timestamp) + uint32(newLockTimeInWeeks * 1
weeks);

stake.weeksNum = newLockTimeInWeeks;

stake.aqualisPower = _calculateAP(stake.amount, newLockTimeInWeeks);

stakers[staker] = stake;

_updateReward(staker, stake.aqualisPower);

emit StakeUpdated(staker, stake.amount, stake.aqualisPower, stake.unlockAt);
}

```

Since the function increments the added staked amount to the already existing stakes before doing the calculation, the following calculation will always return the `remainingLockTime`.

```

uint32 newLockTime =
    uint32((initialStakeTime - remainingLockTime) * (_amount / stake.amount))
+ remainingLockTime;

uint8 newLockTimeInWeeks = uint8(newLockTime / 1 weeks + 1);

```

This is caused by the way solidity handles divisions and decimal numbers. Since `stake.amount` is greater than `_amount`, their division always returns 0. Then the function calculates the `newLockTimeInWeeks` by adding 1 to `newLockTime`. This results in the function always increasing the `remainingLockTime` by 1 week.

Proof of Concept

Add the following test in `AqualisStaking.t.sol` and run it.

```
function test_newLockTimeIsRemaningLockTimePlusOne() external {
```

```

_createPrankStake(staker1, 100, 30);

skip(5 * 1 weeks);

uint256 oldLockTimeInWeeks = staking.getRemainingLockTimeInWeeks(staker1);

vm.startPrank(staker1);

token.approve(address(staking), 900 * 1e18);

staking.increaseStakingAmount(900 * 1e18);

vm.stopPrank();

uint256 newLockTimeInWeeks = staking.getRemainingLockTimeInWeeks(staker1);

        (uint256 newStake, uint128 newAP, uint32 newUnlockAt,) =
staking.getStakeInfo(staker1);

assertEq(oldLockTimeInWeeks + 1, newLockTimeInWeeks);

}

```

Impact

Function does not work as intended. Users that increase their staking amount can inflate their [AP](#) without having to lock their tokens for an extended amount of time.

Recommendation

ABDKMath64x64 for decimals precision or change the calculations to avoid the division returning 0.

Status

Resolved

[H-02] AqualisStaking.sol::stakeFor - Function does not check if users stake already exists.

Description

The [stakeFor](#) function allows the owner to stake for other users. However, the lack of checks causes this function to be vulnerable.

Vulnerability Details

The `stakeFor` function lacks any checks to see if the staked-for user has an existing stake. When this function is called it will override the user's stake if they already are staked. All other staking functions in the contract check if the user has an existing stake.

```
function stakeFor(address _staker, uint256 _amount, uint8 _weeksNum) external  
onlyOwner {  
    aqualisToken.transferFrom(_msgSender(), address(this), _amount);  
    _createStake(_staker, _amount, _weeksNum);  
}
```

Impact

Malicious owner can override any user's existing stake to essentially delete it, this creates a centralization risk.

Additionally, the owner can mistakenly override a user's existing stake.

Recommendation

Implement the following check into the `stakeFor` function that is shown with ++.

```
function stakeFor(address _staker, uint256 _amount, uint8 _weeksNum) external  
onlyOwner {  
    require(stakers[_staker].amount == 0, "User already has active staking");  
    aqualisToken.transferFrom(_msgSender(), address(this), _amount);  
    _createStake(_staker, _amount, _weeksNum);  
}
```

Status

Resolved

Medium

[M-01] AqualisStaking.sol::setMaximumWeeksNum - No upper limit for maximum weeks causes unexpected contract behaviour

Description

The `setMaximumWeeksNum` function allows the owner to change the maximum number of weeks users can stake for. This function needs additional checks.

Vulnerability Details

The `setMaximumWeeksNum` function not requiring an upper limit can cause the calls to the `unstakeWithPenalty` function to revert due to underflow.

```
function setMaximumWeeksNum(uint8 _maximumWeeksNum) external onlyOwner {
    if (_maximumWeeksNum == 0x0) revert ZeroAmount();
    maximumWeeksNum = _maximumWeeksNum;
}
```

As shown above, this function updates the `maximumWeeksNum` variable. This variable used to set an upper limit on the amount on how long the users can stake for.

In an example scenario where the `maximumWeeksNum` variable is set to 150, and the `penaltyPerWeek` variable set to 10 (This can be set as high as 999). Users that unstake their tokens with penalty while their `remainingLockTimeInWeek` is more than 100 will cause the calls to the `unstakeWithPenalty` function to revert due to underflow.

```
function unstakeWithPenalty(uint256 _amount) external nonReentrant {
    //rest of the function...
    uint256 remainingWeeks = getRemainingLockTimeInWeeks(staker);
    require(remainingWeeks > 0, "Stake Matured");
    /// @dev 1% + penaltyPerWeek * remaining weeks (using PERCENT_FRACTION)
    uint256 penaltyPercent = 10 + penaltyPerWeek * remainingWeeks;
    /// @dev penalty divided by 1e3 (PERCENT_FRACTION denominator)
```



```

uint256 penalty = (_amount * penaltyPercent) / 1000;

uint256 returnAmount = _amount - penalty;

//rest of the function...

}

```

This function in the given scenario will calculate the `penaltyPercent` as $10 + 10 * 100 = 1010$. Then the function calculates the `penalty` as $(_amount * 1010) / 1000$. This results in the following condition: `penalty > _amount`.

When the function tries to calculate the `returnAmount`, this will revert due to underflow.

Proof of Concept

Add the following test in `AqualisStaking.t.sol` and run it.

```

function test_MaxWeekCanBreakContract() external {
    vm.prank(owner);
    staking.setPenaltyPerWeek(10);
    _createPrankStake(staker1, 100, 150);
    skip(5 * 1 weeks);
    vm.expectRevert();
    vm.prank(staker1);
    staking.unstakeWithPenalty(100);
}

```

Impact

Unchecked `maximumWeeksNum` amount will break the `unstakeWithPenalty` function.

Recommendation

Add an upper limit to the `maximumWeeksNum` with the changes to the function as shown below:

```

function setMaximumWeeksNum(uint8 _maximumWeeksNum) external onlyOwner {
    if (_maximumWeeksNum == 0x0) revert ZeroAmount();
    require(_maximumWeeksNum * penaltyPerWeek <= 990);
}

```

```
maximumWeeksNum = _maximumWeeksNum;  
}
```

Status

Resolved

[M-02] Contracts - Centralization risk for privileged functions

Description

Contracts with privileged functions need the owner/admin or privileged roles to be trusted not to perform malicious updates.

Vulnerability Details

Following owner or privileged role only functions updates critical variables that will impact user funds:

```
function setRewardsPerWeek()  
function setPenaltyPerWeek()
```

Impact

Malicious privileged roles can change values that will impact user funds. Even in the situation where the privileged roles are not malicious, accidental inputs have a critical impact on users.

Recommendation

To mitigate centralization risks associated with privileged functions, consider implementing a multi-signature or decentralised governance mechanism.

Alternatively, set stricter min/max limits on these functions.

Status

Resolved

Low

[L-01] Contracts - Missing checks for address(0) in constructor

Description

Following variables can not be changed. It is good practice to implement zero address checks in the constructor while setting them.

```
rewardToken = ERC20(_rewardTokenAddress);  
aqualisToken = AQLToken(_aqualisTokenAddress);
```

Recommendation

It is strongly recommended to implement checks to prevent the zero address from being set during the initialization of contracts. This can be achieved by adding require statements that ensure address parameters are not the zero address.

Status

Resolved

[L-02] Contracts - Use Ownable2Step rather than Ownable

Description

Ownable2Step and Ownable2StepUpgradeable prevent the contract ownership from mistakenly being transferred to an address that cannot handle it (e.g. due to a typo in the address), by requiring that the recipient of the owner permissions actively accept via a contract call of its own.

Recommendation

Consider using Ownable2Step or Ownable2StepUpgradeable from OpenZeppelin Contracts to enhance the security of your contract ownership management.

Status

Resolved

[L-03] AqualisStaking.sol::unstake - Function can be optimised to not revert

Description

This function will revert if the user inputted amount is greater than the amount they have staked.

```
function unstake(uint256 _amount) external nonReentrant {
    //rest of the function...
    require(stake.amount >= _amount, "Exceed Staked Amount");
    //rest of the function...
```

This revert can be optimised so that it does not revert but instead calls the [unstakeAll](#) function.

Recommendation

Change the require statement as shown below:

```
function unstake(uint256 _amount) external nonReentrant {
    //rest of the function...
    if(stake.amount < _amount) {
        unstakeAll();
    } else { //continue with the rest of the function
```

Status

Resolved

[L-04] AqualisStaking.sol::unstakeWithPenalty - Function can be optimised to not revert

Description

This function will revert if the user's stake is matured.

```
function unstakeWithPenalty(uint256 _amount) external nonReentrant {
    //rest of the function...
```

```
uint256 remainingWeeks = getRemainingLockTimeInWeeks(staker);

require(remainingWeeks > 0, "Stake Matured");

//rest of the function...
```

This revert can be optimised so that it does not revert but instead calls the **unstake** function.

Recommendation

Change the require statement as shown below:

```
function unstakeWithPenalty(uint256 _amount) external nonReentrant {
    //rest of the function...
    uint256 remainingWeeks = getRemainingLockTimeInWeeks(staker);
    if(remainingWeeks == 0) {
        unstake(_amount);
    } else { //continue with the rest of the function
```

Status

Resolved

[L-05] AqualisStakingReward.sol - Contract lacks a way to distribute unused reward tokens

Description

In case of protocol becoming inactive, there are no functions to retrieve reward tokens stuck in the contract. It is advised to have a way to distribute leftover rewards or withdraw them in unusual circumstances.

Recommendation

Make the reward percentage of an epoch upgradable so that the leftover reward tokens can be distributed.

Alternatively, implement a new function to sweep leftover reward tokens in the contract. The centralization risk this function will create, can be solved by making this function only callable in unusual circumstances such as multiple epochs passing with no activity.

Status

Resolved

Gas

[G-01] AqualisStaking.sol::getRemainingLockTime - Multiple accesses of a mapping/array should use a local variable cache

Description

The instance below points to the second access of a value inside a mapping/array, within a function. Caching a mapping's value in a local storage or calldata variable when the value is accessed multiple times, saves ~42 gas per access.

```
return stakers[_address].unlockAt
```

Recommendation

When a mapping value is accessed multiple times within a function, cache it in a local storage or calldata variable. This approach minimises the gas cost by reducing the number of hash computations for mappings.

Status

Resolved

[G-02] **AqualisStakingReward.sol** - State variables only set in the constructor should be declared immutable

Description

One instance of this is shown below:

```
ERC20 public rewardToken;
```

Recommendation

Consider marking state variables as immutable that never changes on the contract.

Status

Resolved

[G-03] **AqualisStakingReward.sol::getAvailableReward** - Increments can be unchecked in for-loops

Description

Newer versions of the Solidity compiler will check for integer overflows and underflows automatically. This provides safety but increases gas costs. When an unsigned integer is guaranteed never to overflow, the unchecked feature of Solidity can be used to save gas costs

Recommendation

Use unchecked math to block overflow/underflow checks to save gas.

Status

Resolved

Centralisation

The Aqualis project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the Aqualis project seems to have a sound and well-tested code base, now that our findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au

#Hashlock.



#Hashlock.

Hashlock Pty Ltd